AI Report

We are highly confident this text is

AI Generated

AI Probability

100%

This number is the probability that the document is AI generated, not a percentage of AI text in the document.

Plagiarism

?

The plagiarism scan was not run for this document. Go to gptzero.me to check for plagiarism.

ECHOPULSE_CLEANED.pdf - 6/26/2026

EchoPulse AI: A Hybrid Local Data Architecture for Multimodal Music Recommendation Integrating WASABI, MSD, and TalkPlay Datasets

No Author Given

No Institute Given

Abstract. Music recommendation has a problem that bugs me personally: emerging artists barely show up in suggestions. It's not about their music quality. Most algorithms just chase popularity signals, creating loops that keep attention on already-famous hits. This paper introduces EchoPulse AI, a locally-deployed multimodal recommender we built to tackle this visibility gap. The system processes lyrical, acoustic, metadata, and conversational data through a Triple-Encoder Transformer architecture, pulling together heterogeneous datasets (WASABI, Million Song Dataset, Lyrics 1950-2019, and TalkPlayData-2) via a five-stage local pipeline. We went with local deployment instead of cloud infrastructure because of cost constraints and internet instability—pretty common around here in Popayán, Colombia. Maybe the toughest part was unifying the Master Table, synthesizing info from multiple sources into a curated dataset of 5,710 tracks with complete multimodal coverage. Experimental evaluation against collaborative filtering, BERT4Rec, and SASRec baselines gave us results that honestly exceeded what we expected. EchoPulse AI hits competitive accuracy (\text{Hit}@10 = 0.312) while boosting Intra-List Diversity by 45.3% and, most importantly, Emerging Artist Discovery by 157.9%. Core retrieval components run at sub-250ms latency on consumer hardware. The current setup has some clear limitations though: it mostly recommends English-language content, needs adaptation for Spanish-language music, and requires more work to become adaptable to arbitrary music databases. Scaling to industrial catalogs and running online user studies are key next steps.

Keywords: Multimodal recommendation · Transformer architecture · Local data lakehouse · Music information retrieval · Cold-start problem · Conversational AI · Generative evaluation

1 Introduction

Digital streaming totally reshaped the music business. Global revenues hit USD 29.6 billion in 2024, showing massive growth in the streaming era [37]. But for independent musicians, this economic boom often feels like an illusion. The first author spent 14 years playing drums in various bands across Colombia, and the

reality on the ground is pretty stark: new artists just don't get recommended. Not because their music lacks quality. Because algorithms systematically favor established acts.

Most platforms still lean heavily on collaborative filtering. They just look at what people already click on. Works fine for global superstars, but creates a brutal cold-start problem for anyone else [24, 36]. If a new track has no play history, the algorithm ignores it. Creates this vicious feedback loop. In Latin American music scenes, the situation's even worse. Local artists aren't just competing with global hits; they're drowning in a sea of algorithmically-generated background music.

Content-based approaches offer a way out. By looking at actual lyrics, acoustic properties, and metadata of a song, it becomes possible to recommend based on what the music actually sounds like [35]. Comprehensive treatments of recommender systems theory [34] provide the foundation where modern Transformer-based approaches are built. The seminal work 'Attention Is All You Need' by Vaswani et al. [1] particularly inspired this project. Its ability to map long-range dependencies makes Transformers ideal for capturing the complex structure of musical content [40].

This paper introduces EchoPulse AI. We built it as a locally-deployed multi-modal recommender within the IMS Research Group at Fundación Universitaria de Popayán. Why local? Simple answer. The research group had a limited budget for cloud services, and internet connectivity in the region is notoriously unstable. Instead of seeing these constraints as a weakness, the team used them to force a design that prioritizes efficiency, reproducibility, and zero cloud dependency.

Three main contributions emerge from this work. First, a hybrid local data architecture is proposed. It follows a five-stage paradigm to merge messy, heterogeneous music datasets without needing AWS or GCP. Second, a Triple-Encoder Transformer is designed. It processes lyrics, audio, and conversational data through specialized modules, backed by strict mathematical formulations. Third, an end-to-end evaluation is provided. Results prove that you can drastically improve diversity and surface emerging artists without tanking accuracy metrics.

Building the Master Table was easily the hardest part of this project. The team had to reconcile completely different ID systems, handle massive amounts of missing data, and force four distinct datasets into a single, clean collection of 5,710 tracks. That data engineering struggle directly shaped the architecture described in the following sections.

The rest of the paper walks through the approach step-by-step. Section 2 covers background on Transformers and local data architectures. Section 3 dives into the five-stage pipeline and model math. Hardware setup and metrics are detailed in Section 4, followed by an honest look at limitations in Section 5. Finally, Section 6 wraps up findings and what they mean for the future of equitable music recommendation.

2 Related Work

2.1 Transformer-Based Recommendation

Back in 2017, Vaswani and colleagues dropped a paper that changed everything. Their Transformer architecture [1] introduced this clever self-attention mechanism:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^{\top}}{\sqrt{d_k}}\right)\mathbf{V} \quad (1)$$

The math looks intimidating at first glance, but the idea's straightforward. You've got query, key, and value matrices ($\mathbf{Q}$, $\mathbf{K}$, $\mathbf{V}$), and the model figures out which parts of the sequence matter most. This works surprisingly well for music because songs have structure—verses, choruses, bridges—that transformers can pick up on.

BERT4Rec [4] took this further with bidirectional encoding. Improved accuracy, sure, but also inherited all the popularity bias from training data. SASRec [3] went a different route with self-attention for sequential recommendation. Both became our baseline comparisons.

2.2 Multimodal Music Recommendation

Here's where things get interesting. You can't just look at one aspect of a song. Lyrics matter, acoustic features matter, metadata matters. Baltrušaitis et al. [40] laid out theoretical groundwork for combining multiple data sources. Tzanetakis and Cook [28] did early work on genre classification that everyone still builds on.

For fusing audio with text, Porcaro et al. [12] explored some techniques. Cross-modal attention mechanisms showed up in vision-language work like ViLBERT [18] and LXMERT [19], and we adapted those ideas for music.

The datasets we used—WASABI [6] and Million Song Dataset [5]—are pretty much the foundation for this kind of research. They let you analyze lyrics, acoustic features, and metadata at scale. For the emotional side, Russell's circumplex model [42] and Eerola's review [30] help map acoustic and lyrical features to emotional dimensions.

2.3 Local Data Architecture for ML Systems

The Lakehouse paradigm is the new hotness in data management. Armbrust et al. [15] showed how you get data lake flexibility with data warehouse transactional capabilities. Technologies like Delta Lake, Apache Iceberg, and Apache Hudi handle ACID transactions over object storage.

But here's the thing: we didn't have cloud infrastructure. Our implementation adapted these principles locally. Used parquet-based storage with manual versioning, schema validation, and snapshot isolation. Pimentel et al. [45] discussed data versioning approaches that informed our design. Result? Reproducibility and traceability without needing AWS or GCP.

EchoPulse AI Multimodal Music Recommender System Architecture

Fig. 1. EchoPulse AI Local Data Architecture: End-to-end pipeline from heterogeneous sources to local deployment with evaluation feedback loop. The five stages—Data Sources, Data Lake, Data Processing, Data Modeling, and Data Product—ensure reproducibility and traceability without cloud dependencies. Source: Own elaboration.

2.4 Conversational Recommendation Systems

TalkPlay [8] uses large language models for music recommendation through natural language. It's cool, but lacks structured multimodal integration and doesn't rigorously evaluate generative components. We extended this by building a dedicated conversational encoder trained on TalkPlayData-2 (around 16,000 structured dialogues) and proposing evaluation metrics for intention-aware recommendation [26, 27].

Retrieval-augmented generation frameworks [16, 17] provided methodological foundation for integrating external knowledge with generative models. This is exactly what we needed for connecting user intent to specific songs.

3 Methodology

3.1 Data Architecture Overview: Five-Stage Local Pipeline

EchoPulse AI follows a five-stage data pipeline. We adapted established MLOps principles for local deployment. Figure 1 shows the end-to-end architecture from heterogeneous data sources to local deployment with an evaluation feedback loop.

3.2 Data Sources and Integration

We integrated four primary data sources. Each one contributes different modalities:

WASABI Dataset [6]: This gave us lyrical content and cultural metadata for over 2 million commercially released songs, covering 1922 to 2022.

Million Song Dataset (MSD) [5]: Here we got 26 continuous acoustic features—tempo (\tau), energy (E), valence (V), danceability (D), loudness (L)—for 280,000 tracks, normalized to [0, 1] [33].

Lyrics 1950-2019 [7]: This contributed thematic and e

FAQs

What is GPTZero?

GPTZero is the leading AI detector for checking whether a document was written by a large language model such as ChatGPT. GPTZero detects AI on sentence, paragraph, and document level. Our model was trained on a large, diverse corpus of human-written and AI-generated text with support for English, Spanish, French, German, and other languages. To date, GPTZero has served over 17 million users around the world, and works with over 100 organizations in education, hiring, publishing, legal, and more.

When should I use GPTZero?

Our users have seen the use of AI-generated text proliferate into education, certification, hiring and recruitment, social writing platforms, disinformation, and beyond. We've created GPTZero as a tool to highlight the possible use of AI in writing text. In particular, we focus on classifying AI use in prose. Overall, our classifier is intended to be used to flag situations in which a conversation can be started (for example, between educators and students) to drive further inquiry and spread awareness of the risks of using AI in written work.

Does GPTZero only detect ChatGPT outputs?

No, GPTZero works robustly across a range of AI language models, including but not limited to ChatGPT, GPT-5, GPT-4, GPT-3, Gemini, Claude, and AI services based on those models.

What are the limitations of the classifier?

The nature of AI-generated content is changing constantly. As such, these results should not be used to punish students. We recommend educators to use our behind-the-scene Writing Reports as part of a holistic assessment of student work. There always exist edge cases with both instances where AI is classified as human, and human is classified as AI. Instead, we recommend educators take approaches that give students the opportunity to demonstrate their understanding in a controlled environment and craft assignments that cannot be solved with AI. Our classifier is not trained to identify AI-generated text after it has been heavily modified after generation (although we estimate this is a minority of the uses for AI-generation at the moment). Currently, our classifier can sometimes flag other machine-generated or highly procedural text as AI-generated, and as such, should be used on more descriptive portions of text.

I'm an educator who has found AI-generated text by my students. What do I do?

Firstly, at GPTZero, we don't believe that any AI detector is perfect. There always exist edge cases with both instances where AI is classified as human, and human is classified as AI. Nonetheless, we recommend that educators can do the following when they get a positive detection: Ask students to demonstrate their understanding in a controlled environment, whether that is through an in-person assessment, or through an editor that can track their edit history (for instance, using our Writing Reports through Google Docs). Check out our list of several recommendations on types of assignments that are difficult to solve with AI.

Ask the student if they can produce artifacts of their writing process, whether it is drafts, revision histories, or brainstorming notes. For example, if the editor they used to write the text has an edit history (such as Google Docs), and it was typed out with several edits over a reasonable period of time, it is likely the student work is authentic. You can use GPTZero's Writing Reports to replay the student's writing process, and view signals that indicate the authenticity of the work.

See if there is a history of AI-generated text in the student's work. We recommend looking for a long-term pattern of AI use, as opposed to a single instance, in order to determine whether the student is using AI.